PROJECT REPORT

EXPERIMENT - 10

Project Statement

Design and simulate a Smart Refrigerator Monitoring System using DHT11 Sensor. The system monitors internal temperature and humidity conditions to improve food preservation and energy efficiency

Submitted by

Abinaya S – 24MER001

Dhiyanasree KK – 24MER008

Puthiyaraj M – 24MER045

Rithick S – 24MER051

SMART R EFRIGERATOR MONITORING SYSTEM USING DHT11 SENSOR

Project Overview

The Smart Refrigerator Monitoring System is an IoT-based monitoring system designed to continuously monitor the internal temperature and humidity of a refrigerator. The system uses an ESP8266 Node MCU as the main controller and a DHT11/DHT22 sensor to measure environmental conditions inside the refrigerator.

When the temperature or humidity exceeds the predefined safe range, the system activates a buzzer to provide an alert. A relay module can also be used to control a refrigerator-related load or cooling mechanism based on the measured conditions.

The system helps improve food preservation, energy efficiency, and refrigerator condition monitoring by providing timely alerts when abnormal conditions occur.

Main Objectives:

The main objectives of the Smart Refrigerator Monitoring System are:

- To continuously monitor refrigerator temperature.
- To measure humidity inside the refrigerator.
- To detect abnormal temperature and humidity conditions.
- To provide an alert using a buzzer during unsafe conditions.
- To control a connected load using a relay module.
- To improve food preservation by maintaining suitable environmental conditions.
- To reduce unnecessary energy consumption through condition-based control.
- To demonstrate an IoT-based refrigerator monitoring prototype using ESP8266.

Problem Statement

Conventional refrigerators generally operate without continuously informing the user about internal temperature and humidity conditions. Temperature fluctuations, excessive humidity, or cooling problems may affect food quality and increase energy consumption.

Therefore, a Smart Refrigerator Monitoring System is proposed using an ESP8266 Node MCU and DHT11/DHT22 sensor. The system continuously monitors the refrigerator's internal environmental conditions and generates an alert when the measured values exceed predefined limits. A relay module can be used for automatic control of a connected cooling or electrical load.

Problems Addressed:

The proposed system addresses the following problems:

- Lack of continuous temperature monitoring.
- Lack of humidity monitoring.
- Delayed detection of abnormal refrigerator conditions.
- Possible food spoilage due to unsuitable temperature.
- Unnecessary energy consumption.
- Lack of automatic condition-based control.
- Absence of an immediate warning mechanism.

Proposed Solution

The proposed system consists of an ESP8266 Node MCU, DHT11/DHT22 sensor, buzzer, and relay module.

The DHT sensor is positioned inside the refrigerator to measure temperature and humidity. The ESP8266 reads the sensor values and compares them with predefined threshold values.

The system operates as follows:

- **Normal condition:** The measured values remain within the specified range and the system operates normally.
- **High-temperature condition:** The buzzer is activated to indicate a possible cooling problem.
- **High-humidity condition:** The system generates an alert.
- **Abnormal condition:** The relay can be activated/deactivated according to the programmed control logic.

System Architecture

The system can be divided into the following sections:

Input Section:

- DHT11/DHT22 temperature and humidity sensor.

Processing Section:

- ESP8266 Node MCU.
- Sensor data processing.
- Threshold comparison.

Alert Section:

- Buzzer for abnormal-condition indication.

Control Section:

- Relay module for switching a connected load.

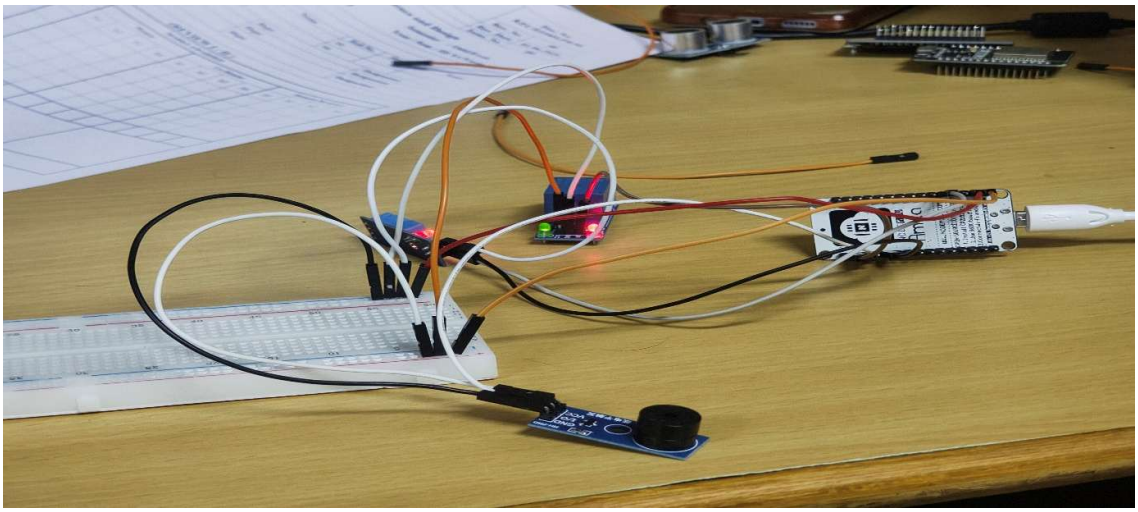
Power Section:

- Suitable power supply through the ESP8266/USB power connection.

Overall Flow:

DHT11/DHT22 Sensor → ESP8266 Node MCU → Data Processing → Threshold Comparison → Buzzer / Relay Control

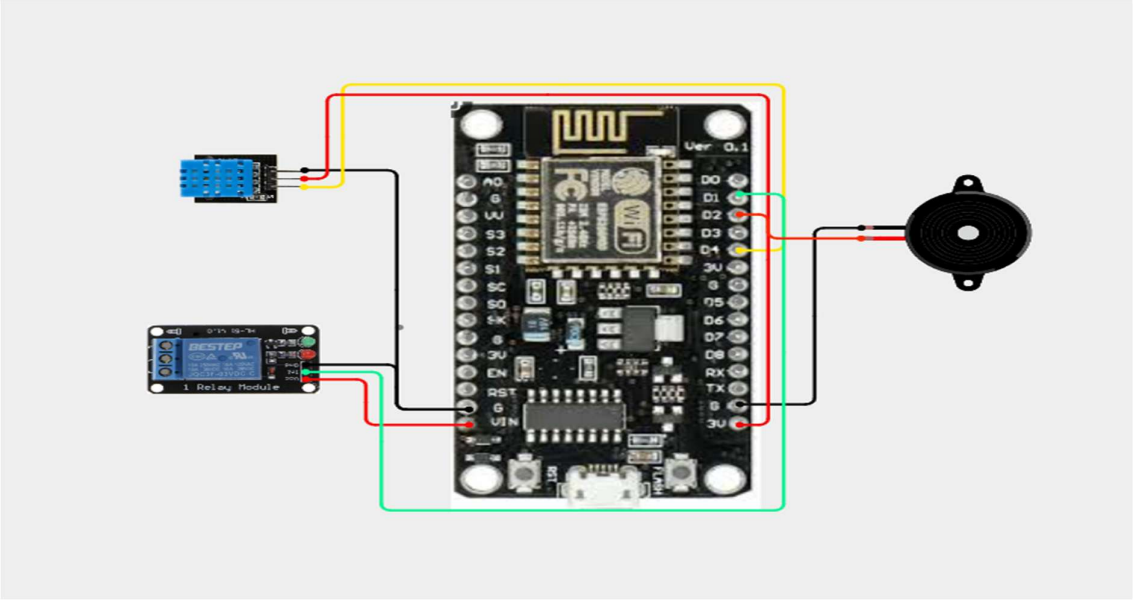
Image:



Circuit Table

S. No	Component	Pin/Terminal	ESP8266 Node MCU Pin
1	DHT11/DHT22 Sensor	VCC	3.3V
		DATA	D2 (GPIO4)
		GND	GND
2	Buzzer	Positive (+)	D5 (GPIO14)
		Negative (-)	GND
3	Relay Module	IN	D6 (GPIO12)
		VCC	5V/VIN*
		GND	GND
4	ESP8266 Node MCU	USB/VIN	Power Supply

Circuit Design



Important Pin Mapping:

- DHT11/DHT22 DATA → D2 (GPIO4)
- Buzzer → D5 (GPIO14)
- Relay IN → D6 (GPIO12)
- DHT VCC → 3.3V
- DHT GND → GND
- Buzzer GND → GND
- Relay GND → GND

Algorithm

- Start the system.
- Initialize the ESP8266 Node MCU.
- Initialize the DHT11/DHT22 sensor.
- Configure the buzzer and relay as output devices.

- Read temperature and humidity values from the DHT sensor.
- Check whether the sensor data is valid.
- Compare the temperature with the predefined threshold.
- Compare the humidity with the predefined threshold.
- If the values are within the acceptable range, keep the system in normal mode.
- If the temperature exceeds the threshold, activate the buzzer.
- If required by the control logic, operate the relay.
- Repeat the monitoring process continuously.

Coding

```
#include <ESP8266WiFi.h>
#include <ThingSpeak.h>
#include <DHT.h>

//====================== WiFi ======================
const char* ssid = "Abinaya";
const char* password = "abinaya1010";

//================== ThingSpeak ==================
unsigned long channelID =  3461083;
const char* writeAPIKey = "RK8KL3F0RTUO5TQT";

WiFiClient client;
```

```
//=================== DHT ==========================
#define DHTPIN 2          // D4
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

//================== Pins ==========================
#define RELAY_PIN 5       // D1
#define BUZZER_PIN 4      // D2

//================= Threshold ======================
float TEMP_LIMIT = 8.0;     // Refrigerator Temperature
float HUM_LIMIT  = 80.0;    // Humidity

//-------------------------------------------------
void setup()
{
  Serial.begin(9600);

  pinMode(RELAY_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(RELAY_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  dht.begin();
```

```
  Serial.println();
  Serial.println("Smart Refrigerator Monitoring System");

  //----------- WiFi ----------
  WiFi.begin(ssid, password);

  Serial.print("Connecting");

  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("WiFi Connected");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  ThingSpeak.begin(client);
}

//-----------------------------------------------
void loop()
{
```

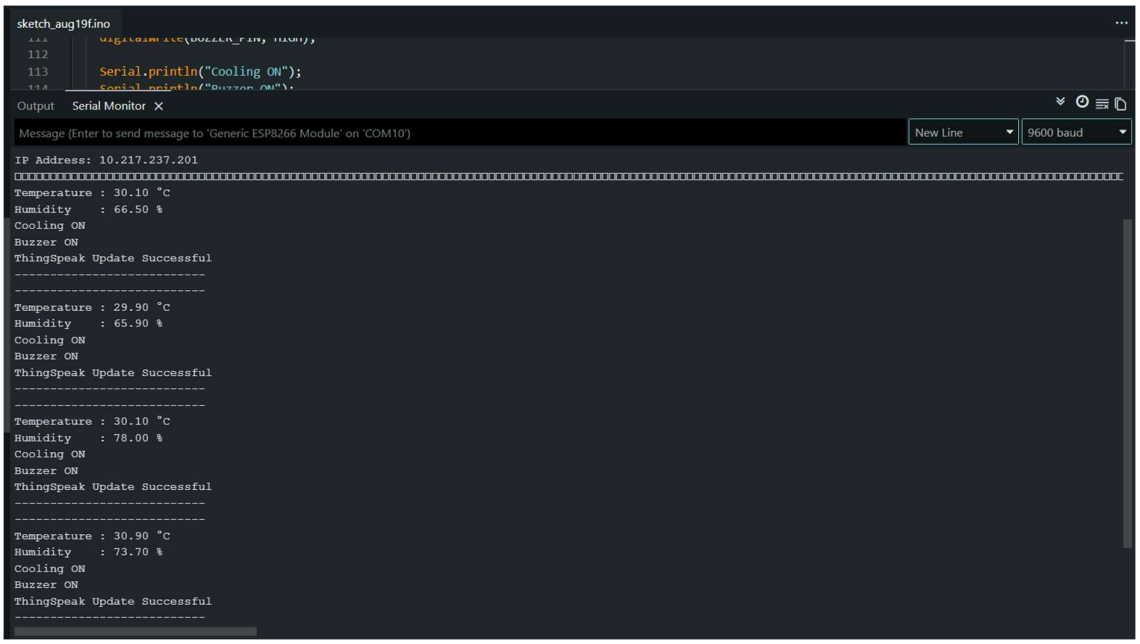
```
// Reconnect WiFi if disconnected
if (WiFi.status() != WL_CONNECTED)
{
  Serial.println("WiFi Lost! Reconnecting...");

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi Reconnected");
}

float humidity = dht.readHumidity();
float temperature = dht.readTemperature();

if (isnan(temperature) || isnan(humidity))
{
  Serial.println("Failed to read DHT Sensor");
  delay(2000);
  return;
}
```

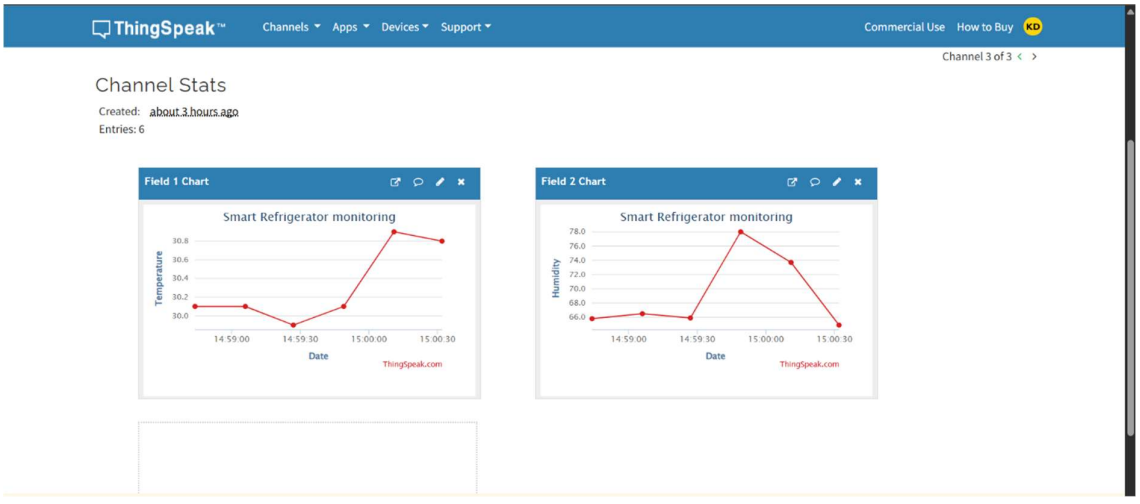
```
Serial.println("--------------------------");
Serial.print("Temperature : ");
Serial.print(temperature);
Serial.println(" °C");

Serial.print("Humidity    : ");
Serial.print(humidity);
Serial.println(" %");

int relayStatus = 0;
int buzzerStatus = 0;

if (temperature > TEMP_LIMIT || humidity > HUM_LIMIT)
{
  relayStatus = 1;
  buzzerStatus = 1;

  digitalWrite(RELAY_PIN, HIGH);
  digitalWrite(BUZZER_PIN, HIGH);

  Serial.println("Cooling ON");
  Serial.println("Buzzer ON");
}
else
{
  relayStatus = 0;
```

```
  buzzerStatus = 0;

  digitalWrite(RELAY_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);

  Serial.println("Cooling OFF");
  Serial.println("Buzzer OFF");
}

//================= Upload to ThingSpeak ==================
ThingSpeak.setField(1, temperature);
ThingSpeak.setField(2, humidity);
ThingSpeak.setField(3, relayStatus);
ThingSpeak.setField(4, buzzerStatus);

int response = ThingSpeak.writeFields(channelID, writeAPIKey);

if (response == 200)
{
  Serial.println("ThingSpeak Update Successful");
}
else
{
  Serial.print("ThingSpeak Error: ");
  Serial.println(response);
}
```

```
  Serial.println("--------------------------");

  // ThingSpeak minimum update interval
  delay(20000);
}
```

Coding Output:





System Working

The system starts when power is supplied to the ESP8266 Node MCU. The DHT11/DHT22 sensor continuously measures the temperature and humidity inside the refrigerator.

The sensor sends the measured data to the ESP8266 through its digital data pin. The ESP8266 processes the received values and compares them with predefined threshold values.

During normal refrigerator conditions, the buzzer remains OFF and the relay operates according to the programmed control logic.

If the temperature rises above the defined safe limit, the ESP8266 detects the abnormal condition and activates the buzzer. This alerts the user that the refrigerator may not be maintaining the required cooling condition.

Similarly, excessive humidity can be detected and used to generate an alert. The relay module can be programmed to control a suitable connected load based on the monitoring conditions.

The process continues repeatedly, allowing continuous monitoring of refrigerator conditions.

Bill of Materials

S. No	Component	Specification	Quantity
1	ESP8266 Node MCU	Wi-Fi-enabled microcontroller	1
2	DHT11/DHT22 Sensor	Temperature & humidity sensor	1
3	Buzzer	5V active buzzer	1
4	Relay Module	1-channel relay module	1
5	Breadboard	Standard solderless breadboard	1
6	Jumper Wires	Male-to-male/male-to-female	1 Set
7	USB Cable	Micro-USB/compatible with Node MCU	1

Prototype Implementation

The prototype is implemented by connecting the DHT11/DHT22 sensor, buzzer, and relay module to the ESP8266 Node MCU.

The DHT sensor is placed in the monitoring area to measure temperature and humidity. The sensor output is connected to a digital GPIO pin of the ESP8266.

The buzzer is connected to another GPIO pin and is used as an alarm indicator. The relay module is connected to a separate GPIO pin and can be used to control a suitable external load.

The complete circuit is assembled on a breadboard using jumper wires. The ESP8266 is programmed to continuously read the sensor values and respond according to predefined threshold conditions.

Hardware Setup:

The hardware setup consists of:

- Place the ESP8266 Node MCU on the breadboard.
- Connect the DHT11/DHT22 sensor to the ESP8266.
- Connect the sensor VCC and GND to the appropriate power and ground pins.
- Connect the DHT DATA pin to D2.
- Connect the buzzer positive terminal to D5 and negative terminal to GND.
- Connect the relay input to D6.
- Connect the relay power and ground according to the relay module specification.
- Connect the ESP8266 to a suitable USB power source.
- Upload the monitoring program to the ESP8266.
- Place the sensor in the refrigerator monitoring area during prototype testing.

Prototype Working

After powering the system, the ESP8266 initializes the DHT11/DHT22 sensor and begins collecting temperature and humidity data.

The measured values are continuously evaluated by the controller.

Normal Condition:

When the measured environmental conditions remain within the programmed limits:

- Buzzer remains OFF.
- System remains in monitoring mode.
- Relay follows the programmed control state.

Abnormal Temperature Condition:

When the temperature rises above the selected threshold:

- ESP8266 detects the abnormal condition.
- Buzzer turns ON.
- User receives a local warning.
- Relay can be operated according to the control program.

Abnormal Humidity Condition:

When humidity exceeds the selected threshold:

- ESP8266 identifies the condition.
- Buzzer can be activated.
- The event can be used for further monitoring or control.

Expected Prototype Output

Condition	Sensor Status	Buzzer	Relay	Expected Output
Normal Condition	Temperature and humidity within limits	OFF	Normal state	Refrigerator condition is normal
High Temperature	Temperature exceeds set limit	ON	Operates according to program	Warning indicates possible cooling problem
High Humidity	Humidity exceeds set limit	ON	Operates according to program	Humidity warning is generated
Temperature & Humidity Normalized	Values return to acceptable range	OFF	Returns to normal state	System resumes normal monitoring
Sensor Reading Error	Invalid/missing sensor data	Alert/diagnostic response	Safe state	Sensor fault is identified

Prototype Demonstration

The prototype can be demonstrated using the following procedure:

- Power ON the ESP8266-based system.
- Allow the DHT sensor to initialize.

- Observe the normal operating condition.
- Monitor the temperature and humidity readings.
- Simulate an increase in temperature around the sensor under controlled test conditions.
- Observe the buzzer response when the programmed threshold is crossed.
- Demonstrate relay operation according to the programmed control condition.
- Return the sensor to normal conditions.
- Verify that the system returns to normal monitoring mode.

Results and Performance Analysis

Results:

The Smart Refrigerator Monitoring System successfully demonstrates continuous monitoring of temperature and humidity using the DHT11/DHT22 sensor and ESP8266 Node MCU.

The system produces the following results:

- The DHT11/DHT22 continuously measures the internal temperature and humidity.
- The ESP8266 successfully receives and processes the sensor data.
- Under normal conditions, the buzzer remains OFF.
- When the temperature exceeds the predefined threshold, the buzzer is activated.
- When humidity exceeds the predefined limit, an alert can be generated.
- The relay module responds according to the programmed control logic.
- The system automatically returns to normal monitoring when the measured values return to the acceptable range.

Performance Analysis:

Performance Parameter	Expected Performance	Analysis
Temperature Monitoring	Continuous	DHT11/DHT22 continuously provides temperature readings
Humidity Monitoring	Continuous	Humidity conditions can be monitored simultaneously
Abnormal Temperature Detection	Threshold-based	System detects temperature exceeding the programmed limit
Abnormal Humidity Detection	Threshold-based	System identifies excessive humidity
Alert Response	Immediate	Buzzer provides a local warning
Relay Response	Automatic	Relay operates according to programmed conditions
Controller Performance	Stable	ESP8266 processes sensor readings effectively
Monitoring Operation	Continuous	System can operate repeatedly without manual intervention
Energy Efficiency	Improved monitoring	Early detection can help identify unnecessary cooling or abnormal operation